

Project Name	Zero Downtime Linux Updates
Online team meeting	https://tu-berlin.zoom-x.de/j/2406759921?pwd=ahCbJqSuz3RajlImkCzVHeSgzqLg2k.1
Production system (if any)	
Test system (if any)	...
GitHub repository	https://github.com/amosproj/amos2026ss01-zero-downtime-linux-updates
GitHub feature board	https://github.com/orgs/amosproj/projects/97
GitHub imp-squared backlog	https://github.com/orgs/amosproj/projects/101
Position of T-shirt logo	Upper Torso Right Small
Link to logo for black T-shirt	https://github.com/amosproj/amos2026ss01-zero-downtime-linux-updates/blob/main/Deliverables/sprint-01/team-logo.svg
Link to logo for white T-shirt	https://github.com/amosproj/amos2026ss01-zero-downtime-linux-updates/blob/main/Deliverables/sprint-01/team-logo.svg
Additional materials	https://discord.gg/cBnpkJKNEu
Team mailing list	oss-amos-proj1@lists.fau.de
Happines Index	https://happy.uni1.de/login
Planning Poker	https://planningpokeronline.com/t2YLAtiYDVOOcJW6gqz/

[illegible]

#	Meeting Day	Product Owner		Software Developer	Release Manager	Scrum Master	Comment
		Review	Planning				
1	2026-04-15	Böhm	Klem	Everyone else	N/A	Patrick Meusling	
2	2026-04-22	Klem	Böhm	Everyone else	Jeremias Friedrich	Patrick Meusling	
3	2026-04-29	Böhm	Klem	Everyone else	Julian Schütz	Patrick Meusling	
4	2026-05-06	Klem	Böhm	Everyone else	Katharina Auer	Patrick Meusling	
5	2026-05-13	Böhm	Klem	Everyone else	Moritz Reinel	Patrick Meusling	
6	2026-05-20	Klem	Böhm	...	Katharina Auer	Patrick Meusling	
7	2026-05-27	Böhm	Klem		Jaroslav Freymann	Patrick Meusling	Mid-term due
8	2026-06-03	Klem	Böhm		Patrick Durand	Patrick Meusling	
9	2026-06-10	Böhm	Klem		Moritz Reinel	Patrick Meusling	
10	2026-06-17	Klem	Böhm		Patrick Durand	Patrick Meusling	
11	2026-06-24	Böhm	Klem		Julian Schütz	Patrick Meusling	
12	2026-07-01	Klem	Böhm		Jeremias Friedrich	Patrick Meusling	
13	2026-07-08	Böhm	Klem		Jaroslav Freymann	Patrick Meusling	
14	2026-07-15	Klem	Böhm		Simeon Zobel	Patrick Meusling	Demo day!
15	2026-07-22	Böhm	Klem		Simeon Zobel	Patrick Meusling	Retrospective
Product owners, software developers, and Scrum Master are set and ideally don't change over time; the critical part is the Release Manager role you need to define here							

Goals	Aquire new skills
	Produce a functioning and valuable product
Meeting norms	We show up on time
	Devs make sure to try to understand the new sprints tickets before the team meeting
	POs provide a summary of industry partner meeting
Working norms	Clean, testable code with clear commit messages, code comments
	We respect other people's work
	PRs must be ready for review by Tuesday 6:00 am. Tuesdays are reserved for review and merging
Coordination norms	Separate branches for issues, branches only get merged via Merge Requests, second person to review
	We balance workload among the team
Communication norms	We regularly check the corresponding channels
	We make sure to communicate possible problems as early as possible (absence, etc.)
	We communicate constructively
Consideration norms	We discuss issues openly
	We vote in case we can't reach a consensus
Cont. improvement norms	We encourage critique and improvement efforts
	We reach out for help early enough to still finish our tasks
Rewards	Favorite Drink
	Wear a cool hat or glasses during meetings
Sanctions	Publish a stackoverflow answer
Signatures	
Scrum Master	Patrick Meusling
Product owner	Luca Klem
Product owner	Luca Böhm
Software developer	Katharina Auer
Software developer	Jaroslav Freymann
Software developer	Julian Schütz
Software developer	Patrick Durand
Software developer	Moritz Reinel
Software developer	Simeon Zobel
Software developer	Jeremias Friedrich

[illegible]

Term	Definition
Edge Device/ Edge IPC	An industrial Inter-Process Communication (IPC) hardware asset deployed on the factory floor on which our project runs.
Edge Agent	The sytem component that is running on the Edge IPC and controls Updates and Communication. This includes Orchestrator, Download manager, and Health checks
Orchestrator	The central, automated controller within the Edge Agent that coordinates the execution of all local update activities. It is responsible for executing the update cycle by interpreting the remote configuration, monitoring the local system status, and sequentially triggering download, verification, and installation workflows to align the physical device with the cloud's target state.
Device Identity	A unique identifier permanently bound to a physical Edge Device, derived directly from its onboard hardware.
Image	A complete representation of a software layer packaged as a containerized asset ready for deployment.
Download Manager	A component within the Edge Agent tasked with fetching the required state of OS and applications on the device from the Cloud.
Inventory Database	Database in the Cloud that stores all information of the Edge Device, Appcations, and OS Versions. On top of that it stores the current deployed and the required OS and application version for each Edge Device.
Rollback	An automated emergency recovery operation that instantly reverts an Edge Device to its previously functional Operating System Image or Application Stack upon a post-update health check failure.
Health Check	A set of automated local test scripts executed immediately after a system switch or reboot to verify the functional correctness of critical device services before confirming an update as successful.
Fleet	The total of all edge devices that are managed by one company/user.
Database	Central Database where information about Edge Devices is gathered. This includes but isnt limited to: OS version, Host Info, Tenant, Group
Tenant	The customer where the Edge Device and its associated machinery is located
Group	Edge Devices can be put into Groups so that they can be modified together. IP has envisioned different Groups like porduction, testing etc.
Cloud	Includes Invetory Database and APIs to communicate with which the user and Edge IPC can communicate.

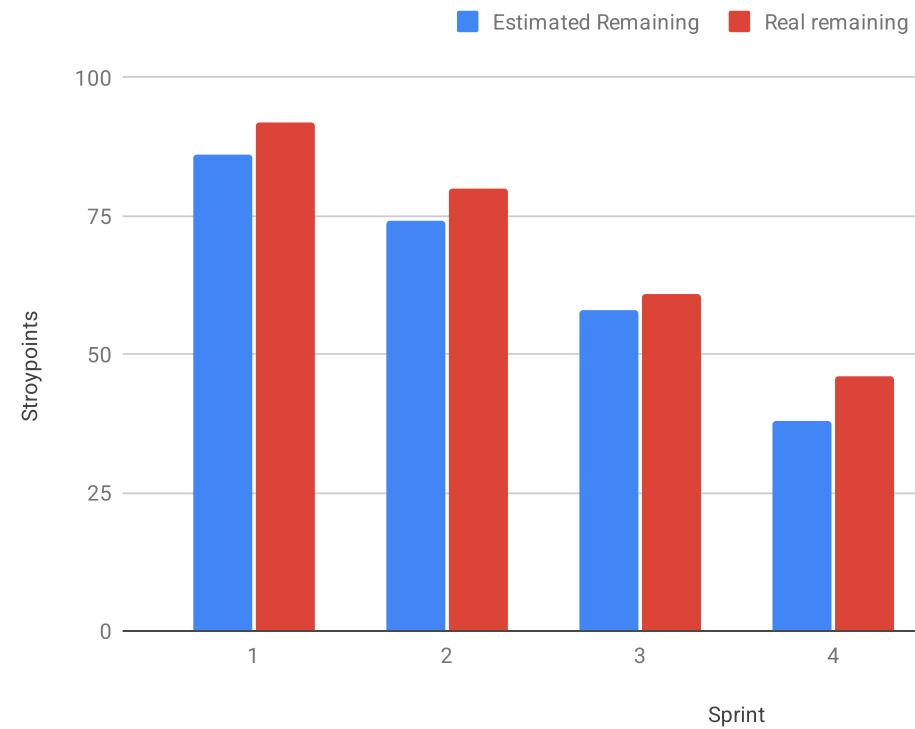
Sprint #	Sprint goal
1	None
2	None
3	None
4	Optional
5	Enable OS update swapping
6	Database interaction with the user and Edge IPC is enabled
7	Enhance database functionality and logging
8	Integration Testing and Test Environment
9	Save device and application Logs in Database
10	Provide Logs and Streaming for Users
11	First Demo preparations
12	Finalize Project and Demo preparation
13	Prepare Handover
14	
15	

Sprint #	Story Points Realized
1	0
2	12
3	19
4	15
5	25
6	21
7	11
8	19
9	24
10	0
11	0
12	0
13	0
	PLEASE CREATE THE VELOCITY CHART ON A NEW TAB USING THE DATA FROM THIS TAB

Sprint	Goal	Feature Name	Est. size	Est. remaining	Real size	Real remaining
Release						
Total			86	86	92	
Sprints						
1	None		0	86	0	92
2	None		12	74	12	80
3	None		16	58	19	61
4	Optional		20	38	15	46
5	Enable OS update swapping		21	17	25	21
6	Database interaction with the user and Edge IPC is enabled		17	0	21	0
Features						
1	None					
2	None	Review different Linux Distros	3		3	
		Review Backup options	2		2	
		Setup shared dev environment	2		2	
		Review options for zero downtime update	2		2	
		Initialize SBOM and Software Architecture	1		1	
		Review how TPM can be used	2		2	
3	None	HTTPS Download Manager	3		3	
		API Protocoll & Mock Cloud	3		5	
		Setup DCO & CI Pipeline	3		5	
		Edge IPC Orchestrator	5		5	
		Signature Verification	2		1	
4	Optional	Setup Cargo Workspace	2		2	
		rpm-ostree OS Update Controller	2		2	
		Post-Update Health Checks	5		3	
		Containerized Image Delivery	8		5	
		Device Inventory MVP	3		3	

Sprint	Goal	Feature Name	Est. size	Est. remaining	Real size	Real remaining
5	Enable	Build Process Video	2		2	
		Orchestrator check for update	3		5	
		Application Stack Controller	3		5	
		Refactor: bootc wrapper	3		3	
		rpm-ostree OS Update Controller	2		2	
		Update Documentation and Architecture	3		3	
		Setup Inventory Database	5		5	
6	Databas	Create Scheme for Database and API calls	3		3	
		Refactor: Orchestrator initiates updates	3		5	
		Edge IPC to Cloud API for Database updates	3		3	
		User to Cloud API for Database Updates	3		5	
		Setup demo review	5		5	
		Refactor: Folder strucuter	1		1	
		PLEASE CREATE THE BURNDOWN CHART ON A NEW TAB USING THE DATA FROM THIS TAB				

Mid-Project Burndown Chart



Sprint	Goal	Feature Name	Est. size	Est. remaining	Real size	Real remaining
Release						
Total			135	135		
Sprints						
7	Enhance database functionality and logging		15	120	11	124
8	Integration Testing and Test Environment		19	101	19	105
9	Save device and application Logs in Database		21	80	24	81
10	Provide Logs and Streaming for Users		24	56	0	81
11	First Demo preparations		23	33	0	81
12	Finalize Project and Demo preperation		18	15	0	81
13	Prepare Handover		15	0	0	81
Features						
7	Enhance database functionality and logging					
		Refactor: Introduce DTO	3		3	
		OS Image creation of different versions	3		2	
		Add Heartbeats	1		1	
		Research: Timestamp for DB changes	3		1	
		Signature Verification of image delivery	3		2	
		Add paging to device/group/application information	2		2	
8	Integration Testing and Test Environment					
		Create test setup	8		8	
		Create Integration tests	5		5	
		Authentication for User API	3		3	
		Research: Solution for storing time sensitive data	3		3	
9	Save device and application Logs in Database					
		Provide device logging function for storing logs in database	2		3	
		TPM-based Device Identity Verification at Edge IPC-Cloud API	5		8	
		Log timestamp and user information for database changes	3		3	
		Monitor and Mange Podman Containers	8		5	
		Create API for time sensitive data	3		5	
10	Provide Logs and Streaming for Users					
		Store App Config in databse	5			
		Store Podman logs in database	3			
		Store Device Logs in Database	2			
		Refactor Orchestrator	2			
		Hardware deployment	3			
		Adapt database to support changelogs in UI	5			
		Testing: Adapt Integration Testing so that TPM works with it	1			
		Testing: Improve test setup so that changing binary in vm is easier	3			
11	First Demo preparations					
		Update Post-Bootting Health checks	2			
		API for uploading TPM/ID pairs and first device registration	3			

[illegible]

#	Feature Definition of Done	Sprint Release Definition of Done	Project Release Definition of Done
	All Acceptance criterias defined in the backlog item are met and verified		
	Work done is pushed to the Github repository		
	For each backlog item a branch is created		
	Pull requests are created for each branch		
	Pull requests are reviewed		
	Corresponding branches are merged		
	Software Bill of Materials in the planning document is updated if new dependencies were added		
	The Wiki Section/Documentation corresponding to the work done is updated		
	Unit tests are written for all new logic		
	All unit tests must pass successfully in the CI pipeline		
	Code passes cargo fmt (style) and cargo clippy (linting)		
	All Developers assigned to a backlog item must communicate and coordinate with each other to complete the task		
		All features merged into the main branch; the CI pipeline is "Green"	
		Release candidates with a working and meaningful update to the previous sprint is tagged	
		Previously established features must continue to work.	
			All automated tests and validation checks for the developed components pass without errors
			A single-click deployment/flashable image for the selected hardware is available
			User documentation (usage examples, configuration steps, expected outputs) is finalized and up to date
			All open-source licenses are documented and compliant
			A "Handover Document" for Weber is completed in the Wiki
			The final release has been reviewed and approved by all project team members and the industry partner

[illegible]

Last Name	First Name	Value					
Auer	Katharina	0					
#REF!	#REF!	0			0.00	OK	
#REF!	#REF!						
#REF!	#REF!						
#REF!	#REF!						
#REF!	#REF!				0	No size	
#REF!	#REF!				1	Trivial size	
#REF!	#REF!				2	Small size	
#REF!	#REF!				3	Medium size	
#REF!	#REF!				5	Large size	
					8	Very large size	
					13	Too large (size)	
How to play planning poker							
1. Everyone type their number into their value field, don't hit return yet							
2. Someone, perhaps a product owner, count down 3.. 2.. 1..							
3. Then, everyone hit return to submit their value							

[illegible]

